

SIMATS ENGINEERING

ASSESSMENT TOOL 2 — ANSWER KEY

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME (CO3): Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool: Case Study (Medium)

Assessment Tool Weightage: 30%

Question 1 (5 Marks)

Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Case analysis

Each order line records a customer buying a product; customer details and product details are duplicated on every line of every order that references them — a classic 3NF/redundancy case in e-commerce systems.

Functional dependencies

- OrderID → CustID (each order belongs to one customer)
- CustID → CustName
- ProdID → ProdName, Price
- OrderID, ProdID → Qty (quantity is specific to a product within an order)

Normalization to 3NF

Relation	Attributes	Key
CUSTOMER	CustID, CustName	CustID
PRODUCT	ProdID, ProdName, Price	ProdID
ORDERS	OrderID, CustID	OrderID
ORDER_ITEM	OrderID, ProdID, Qty	OrderID, ProdID

```
CREATE TABLE CUSTOMER(CustID TEXT PRIMARY KEY, CustName TEXT);
INSERT INTO CUSTOMER VALUES ('C01','Meena'),('C02','Ajay');

CREATE TABLE PRODUCT(ProdID TEXT PRIMARY KEY, ProdName TEXT, Price REAL);
INSERT INTO PRODUCT VALUES ('P01','Mouse',450.0),('P02','Keyboard',900.0);

CREATE TABLE ORDERS(OrderID INT PRIMARY KEY, CustID TEXT REFERENCES CUSTOMER(CustID));
INSERT INTO ORDERS VALUES (1001,'C01'),(1002,'C02');

CREATE TABLE ORDER_ITEM(OrderID INT, ProdID TEXT, Qty INT, PRIMARY KEY(OrderID,ProdID));
INSERT INTO ORDER_ITEM VALUES (1001,'P01',2),(1001,'P02',1),(1002,'P01',5);

SELECT o.OrderID, c.CustName, oi.ProdID, p.ProdName, oi.Qty, p.Price
FROM ORDERS o JOIN CUSTOMER c ON o.CustID=c.CustID
JOIN ORDER_ITEM oi ON o.OrderID=oi.OrderID
JOIN PRODUCT p ON oi.ProdID=p.ProdID;
```

Output:

OrderID	CustName	ProdID	ProdName	Qty	Price
1001	Meena	P01	Mouse	2	450.0
1002	Ajay	P01	Mouse	5	450.0
1001	Meena	P02	Keyboard	1	900.0

Conclusion: Customer name is stored once regardless of how many orders they place, and product name/price is stored once regardless of how many orders include it — both classic e-commerce redundancy sources are removed.

Question 2 (5 Marks)

A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Anomalies in the flat schema

- Update anomaly: if Dr. Nair's specialization is corrected, every patient row under Dr. Nair must be updated.
- Insertion anomaly: a new doctor cannot be added to the roster until they are assigned at least one patient.
- Deletion anomaly: discharging the last patient in 'Ward W2' erases all record that Ward W2 (its name) exists.

Functional dependencies

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Both DocID→(DocName,Specialization) and WardNo→WardName are determinants that are not super keys of PATIENT, violating BCNF (and 3NF).

BCNF decomposition

```
CREATE TABLE DOCTOR(DocID TEXT PRIMARY KEY, DocName TEXT, Specialization TEXT);
INSERT INTO DOCTOR VALUES ('D01','Dr. Nair','Cardiology'),('D02','Dr. Rekha','Orthopedics');

CREATE TABLE WARD(WardNo TEXT PRIMARY KEY, WardName TEXT);
INSERT INTO WARD VALUES ('W1','ICU'),('W2','General');

CREATE TABLE PATIENT(PID INT PRIMARY KEY, PName TEXT,
                      DocID TEXT REFERENCES DOCTOR(DocID),
                      WardNo TEXT REFERENCES WARD(WardNo));
INSERT INTO PATIENT VALUES (1,'Ramesh','D01','W1'),(2,'Sita','D02','W2');

SELECT p.PID, p.PName, d.DocName, d.Specialization, w.WardName
FROM PATIENT p JOIN DOCTOR d ON p.DocID=d.DocID JOIN WARD w ON p.WardNo=w.WardNo;
```

Output:

PID	PName	DocName	Specialization	WardName
1	Ramesh	Dr. Nair	Cardiology	ICU
2	Sita	Dr. Rekha	Orthopedics	General

Conclusion: In PATIENT, DOCTOR and WARD, every determinant (PID, DocID, WardNo respectively) is a super key of its own relation, so all three relations satisfy BCNF.

Question 3 (5 Marks)

Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Functional dependencies

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $SID, CourseID \rightarrow Grade$

Candidate key: $\{SID, CourseID\}$. $SName$ and $(CourseName, Faculty)$ each depend on only part of the key — SID and $CourseID$ respectively — so both are partial dependencies, violating 2NF.

2NF decomposition

Relation	Attributes	Key
STUDENT	$SID, SName$	SID
COURSE	$CourseID, CourseName, Faculty$	$CourseID$
ENROLL	$SID, CourseID, Grade$	$SID, CourseID$

```
CREATE TABLE STUDENT(SID INT PRIMARY KEY, SName TEXT);
INSERT INTO STUDENT VALUES (1, 'Farah'), (2, 'Om');

CREATE TABLE COURSE(CourseID TEXT PRIMARY KEY, CourseName TEXT, Faculty TEXT);
INSERT INTO COURSE VALUES ('CS01', 'DBMS', 'Dr. Iyer'), ('CS02', 'OS', 'Dr. Bose');

CREATE TABLE ENROLL(SID INT, CourseID TEXT, Grade TEXT, PRIMARY KEY(SID, CourseID));
INSERT INTO ENROLL VALUES (1, 'CS01', 'A'), (1, 'CS02', 'B'), (2, 'CS01', 'A');

SELECT s.SName, c.CourseName, c.Faculty, e.Grade
FROM ENROLL e JOIN STUDENT s ON e.SID=s.SID JOIN COURSE c ON e.CourseID=c.CourseID;
```

Output:

SName	CourseName	Faculty	Grade
Farah	DBMS	Dr. Iyer	A
Farah	OS	Dr. Bose	B
Om	DBMS	Dr. Iyer	A

Conclusion: With the partial dependencies removed, STUDENT and COURSE now each store their descriptive attributes once. Since no transitive dependency remains either, this decomposition already satisfies 3NF as well.

Question 4 (5 Marks)

Given the airline booking case: $Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)$. Normalize up to BCNF.

Functional dependencies

- $FlightNo, Date \rightarrow DeptCity, ArrCity$ (a flight number on a given date has one route)
- $FlightNo, Date, Seat \rightarrow PassID$ (a seat on a specific flight/date is occupied by one passenger)
- $PassID \rightarrow PassName$

Candidate key: $\{FlightNo, Date, Seat\}$. $PassID \rightarrow PassName$ is a determinant ($PassID$) that is not a super key, violating BCNF.

BCNF decomposition

```
CREATE TABLE FLIGHT(FlightNo TEXT, Date TEXT, DeptCity TEXT, ArrCity TEXT,
                    PRIMARY KEY(FlightNo, Date));
INSERT INTO FLIGHT VALUES ('AI202', '2026-08-20', 'Chennai', 'Delhi');

CREATE TABLE PASSENGER(PassID TEXT PRIMARY KEY, PassName TEXT);
```

```

INSERT INTO PASSENGER VALUES ('PS01','Kavya'),('PS02','Rohit');

CREATE TABLE BOOKING(FlightNo TEXT, Date TEXT, Seat TEXT, PassID TEXT,
                      PRIMARY KEY(FlightNo,Date,Seat));
INSERT INTO BOOKING VALUES ('AI202','2026-08-20','12A','PS01'),
                           ('AI202','2026-08-20','12B','PS02');

SELECT b.FlightNo, f.DeptCity, f.ArrCity, b.Date, b.Seat, p.PassName
FROM BOOKING b JOIN FLIGHT f ON b.FlightNo=f.FlightNo AND b.Date=f.Date
JOIN PASSENGER p ON b.PassID=p.PassID;

```

Output:

FlightNo	DeptCity	ArrCity	Date	Seat	PassName
AI202	Chennai	Delhi	2026-08-20	12A	Kavya
AI202	Chennai	Delhi	2026-08-20	12B	Rohit

Conclusion: FLIGHT, PASSENGER and BOOKING each have a determinant equal to their key, so all three are in BCNF; route information is stored once per flight/date, and a passenger's name is stored once regardless of how many flights they book.

Question 5 (5 Marks)

Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Given (flat) schema

EMPLOYEE_HR(EmpID, EmpName, DeptID, DeptName, CountryCode, CountryName, Salary)

Functional dependencies

- EmpID → EmpName, DeptID, Salary
- DeptID → DeptName, CountryCode
- CountryCode → CountryName

Transitive dependencies: EmpID → DeptID → DeptName (via DeptID) and EmpID → DeptID → CountryCode → CountryName (a two-level transitive chain). Both violate 3NF because non-prime attributes (DeptName, CountryCode, CountryName) depend on the key only through other non-prime attributes.

Normalization to 3NF

Relation	Attributes	Key
COUNTRY	CountryCode, CountryName	CountryCode
DEPARTMENT	DeptID, DeptName, CountryCode	DeptID
EMPLOYEE	EmpID, EmpName, DeptID, Salary	EmpID

```

CREATE TABLE COUNTRY(CountryCode TEXT PRIMARY KEY, CountryName TEXT);
INSERT INTO COUNTRY VALUES ('IN','India'),('SG','Singapore');

CREATE TABLE DEPARTMENT(DeptID INT PRIMARY KEY, DeptName TEXT,
                        CountryCode TEXT REFERENCES COUNTRY(CountryCode));
INSERT INTO DEPARTMENT VALUES (1,'Finance','IN'),(2,'Engineering','SG');

CREATE TABLE EMPLOYEE(EmpID INT PRIMARY KEY, EmpName TEXT,
                        DeptID INT REFERENCES DEPARTMENT(DeptID), Salary REAL);
INSERT INTO EMPLOYEE VALUES (1,'Neha',1,75000),(2,'Wei',2,98000);

SELECT e.EmpName, d.DeptName, c.CountryName, e.Salary

```

```
FROM EMPLOYEE e JOIN DEPARTMENT d ON e.DeptID=d.DeptID
JOIN COUNTRY c ON d.CountryCode=c.CountryCode;
```

Output:

EmpName	DeptName	CountryName	Salary
Neha	Finance	India	75000
Wei	Engineering	Singapore	98000

Conclusion: Each transitive chain is broken at its own level (Department depends on Employee only through DeptID; Country depends on Department only through CountryCode), so all three relations reach 3NF with no repeated country or department data.

Question 6 (5 Marks)

Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Analysis

A member borrows several books over time, and independently lists several preferred genres in their profile — the set of borrowed books and the set of preferred genres vary independently for the same member, which is the signature of a multi-valued dependency (not an FD):

- MemberID $\twoheadrightarrow$ BookID (borrow history, paired with BorrowDate)
- MemberID $\twoheadrightarrow$ Genre (independent preference list)

Storing both multi-valued facts in one row forces every (BookID, Genre) pair to be duplicated for each member — redundancy that neither 1NF/2NF/3NF nor BCNF catches, because no FD is violated.

4NF decomposition

```
CREATE TABLE MEMBER(MemberID INT PRIMARY KEY, MemberName TEXT);
CREATE TABLE BOOK(BookID TEXT PRIMARY KEY, Title TEXT, Author TEXT);

CREATE TABLE BORROW(MemberID INT, BookID TEXT, BorrowDate TEXT,
    PRIMARY KEY(MemberID,BookID,BorrowDate));

CREATE TABLE MEMBER_GENRE(MemberID INT, Genre TEXT, PRIMARY KEY(MemberID,Genre));
INSERT INTO MEMBER_GENRE VALUES (1,'Fiction'),(1,'Sci-Fi'),(2,'Mystery');

INSERT INTO BORROW(MemberID,BookID) VALUES (1,'B01'),(1,'B02'),(2,'B01');
SELECT * FROM BORROW;
SELECT * FROM MEMBER_GENRE;
```

Output:

MemberID	BookID
1	B01
1	B02
2	B01

MemberID	Genre
1	Fiction
1	Sci-Fi
2	Mystery

Conclusion: BORROW and MEMBER_GENRE each carry only one multi-valued fact per member, so both are in 4NF; book borrowing history no longer needs to be cross-multiplied against genre preferences.

Question 7 (5 Marks)

Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Given (unnormalized) schema

BILL(BillID, CustID, CustName, PlanID, PlanName, PlanRate, MonthYear, PhoneNo, Amount)

Functional dependencies

- BillID → CustID, PlanID, MonthYear, PhoneNo, Amount (a bill is a single billing event)
- CustID → CustName, PhoneNo (each customer has one registered name and number)
- PlanID → PlanName, PlanRate
- CustID, MonthYear → BillID (a customer receives at most one bill per month — alternate identification)

Candidate key analysis

Candidate set	Closure reasoning	Candidate key?
{BillID}	BillID→everything else directly or transitively	Yes
{CustID, MonthYear}	→BillID, and BillID→ everything else	Yes
{CustID}	Does not determine BillID/MonthYear/Amount alone	No
{PlanID}	Only determines PlanName, PlanRate	No

Two candidate keys exist: {BillID} and {CustID, MonthYear}. BillID is chosen as the primary key since it is a single surrogate attribute already used as the natural billing-system identifier; {CustID, MonthYear} remains a unique (alternate) key enforceable with a UNIQUE constraint.

Conclusion: Once the FDs and keys are catalogued this way, the next normalization step (not required by this question) would separate CUSTOMER(CustID,CustName,PhoneNo), PLAN(PlanID,PlanName,PlanRate) and BILL(BillID,CustID,PlanID,MonthYear,Amount) to remove the transitive redundancy of customer and plan details repeating on every bill.

Question 8 (5 Marks)

A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Functional dependencies

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Key: {PID}. SupplierID→SupplierName is transitive (PID→SupplierID→SupplierName), violating 3NF/BCNF. Category, Price and Warehouse are each single-valued per product with no further multivalued or join dependency indicated, so once the transitive dependency is removed the design reaches BCNF (which, with only these FDs, coincides with the highest normal form applicable here).

Complete normalization

```
CREATE TABLE SUPPLIER(SupplierID TEXT PRIMARY KEY, SupplierName TEXT);
INSERT INTO SUPPLIER VALUES ('S01','ABC Traders'),('S02','XYZ Supplies');

CREATE TABLE PRODUCT(PID TEXT PRIMARY KEY, PName TEXT,
                      SupplierID TEXT REFERENCES SUPPLIER(SupplierID),
                      Category TEXT, Price REAL, Warehouse TEXT);
INSERT INTO PRODUCT VALUES
('P100','LED Bulb','S01','Electrical',120.0,'WH-1'),
('P101','Notebook','S02','Stationery',40.0,'WH-2');
```

```
SELECT p.PID, p.PName, s.SupplierName, p.Category, p.Price, p.Warehouse
FROM PRODUCT p JOIN SUPPLIER s ON p.SupplierID=s.SupplierID;
```

Output:

PID	PName	SupplierName	Category	Price	Warehouse
P100	LED Bulb	ABC Traders	Electrical	120.0	WH-1
P101	Notebook	XYZ Supplies	Stationery	40.0	WH-2

Conclusion: SupplierName is now stored once per supplier instead of once per product, so a supplier renaming touches a single row in SUPPLIER; PRODUCT and SUPPLIER are both in BCNF.

Question 9 (5 Marks)

Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Given (flat) schema

SCHOOL_ERP(StudID, StudName, ClassID, ClassSection, TeacherID, TeacherName)

Functional dependencies

- StudID → StudName, ClassID
- ClassID → ClassSection, TeacherID
- TeacherID → TeacherName

Two chained transitive dependencies exist: StudID→ClassID→ClassSection/TeacherID, and ClassID→TeacherID→TeacherName. Neither ClassID nor TeacherID is a super key of the flat relation, so it violates both 3NF and BCNF.

Proposed BCNF design

Relation	Attributes	Key	Justification
TEACHER	TeacherID, TeacherName	TeacherID	TeacherID is the sole determinant → super key
CLASS	ClassID, ClassSection, TeacherID	ClassID	ClassID determines both remaining attributes
STUDENT	StudID, StudName, ClassID	StudID	StudID determines both remaining attributes

```
CREATE TABLE TEACHER(TeacherID TEXT PRIMARY KEY, TeacherName TEXT);
INSERT INTO TEACHER VALUES ('T01', 'Mr. Das'), ('T02', 'Ms. Roy');

CREATE TABLE CLASS(ClassID TEXT PRIMARY KEY, ClassSection TEXT,
                    TeacherID TEXT REFERENCES TEACHER(TeacherID));
INSERT INTO CLASS VALUES ('C10A', '10-A', 'T01'), ('C10B', '10-B', 'T02');

CREATE TABLE STUDENT(StudID INT PRIMARY KEY, StudName TEXT,
                     ClassID TEXT REFERENCES CLASS(ClassID));
INSERT INTO STUDENT VALUES (1, 'Tanvi', 'C10A'), (2, 'Ishaan', 'C10B');

SELECT s.StudName, c.ClassSection, t.TeacherName
FROM STUDENT s JOIN CLASS c ON s.ClassID=c.ClassID
JOIN TEACHER t ON c.TeacherID=t.TeacherID;
```

Output:

StudName	ClassSection	TeacherName
----------	--------------	-------------

Tanvi	10-A	Mr. Das
Ishaan	10-B	Ms. Roy

Conclusion: In each of TEACHER, CLASS and STUDENT the left-hand side of every FD is that relation's key, so all three satisfy BCNF; a class-teacher reassignment now touches a single row in CLASS instead of every enrolled student's row.

Question 10 (5 Marks)

Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Case

STREAM(User, Movie, Device) records that a user watches certain movies on certain devices. Suppose the platform's rule is circular: 'if user U watches movie M, and movie M is available on device D, and user U owns device D, then U watches M specifically on D.' This is a join dependency $*\{(User, Movie), (Movie, Device), (User, Device)\}$ — it cannot be correctly captured by any 2 of the 3 pairwise projections; the JD requires 5NF decomposition into all three.

```
CREATE TABLE STREAM(User TEXT, Movie TEXT, Device TEXT);
INSERT INTO STREAM VALUES
  ('U1', 'M1', 'Mobile'), ('U1', 'M2', 'Mobile'), ('U2', 'M1', 'Mobile'), ('U1', 'M1', 'TV');
SELECT * FROM STREAM; -- original: redundant
```

Output:

User	Movie	Device
U1	M1	Mobile
U1	M2	Mobile
U2	M1	Mobile
U1	M1	TV

5NF decomposition into three binary projections

```
CREATE TABLE USER_MOVIE(User TEXT, Movie TEXT, PRIMARY KEY(User,Movie));
INSERT INTO USER_MOVIE SELECT DISTINCT User,Movie FROM STREAM;

CREATE TABLE MOVIE_DEVICE(Movie TEXT, Device TEXT, PRIMARY KEY(Movie,Device));
INSERT INTO MOVIE_DEVICE SELECT DISTINCT Movie,Device FROM STREAM;

CREATE TABLE USER_DEVICE(User TEXT, Device TEXT, PRIMARY KEY(User,Device));
INSERT INTO USER_DEVICE SELECT DISTINCT User,Device FROM STREAM;

-- Lossless-join verification: 3-way natural join must equal the original STREAM
SELECT um.User, um.Movie, md.Device
FROM USER_MOVIE um JOIN MOVIE_DEVICE md ON um.Movie=md.Movie
JOIN USER_DEVICE ud ON um.User=ud.User AND md.Device=ud.Device
ORDER BY um.User, um.Movie, md.Device;
```

Output:

User	Movie	Device
U1	M1	Mobile
U1	M1	TV
U1	M2	Mobile
U2	M1	Mobile

Conclusion: The 3-way join reproduces exactly the 4 original rows — no extra (spurious) rows and none missing — confirming a lossless-join 5NF decomposition. (Verification note: joining only 2 of the 3 projections, e.g. USER_MOVIE ⋈ MOVIE_DEVICE alone, introduces a spurious row not present in STREAM, which is precisely why all three projections are required by the join dependency.)